

GUI 与控制模块详细设计

一 模块概述

GUI 与控制模块是用户直接接触的部分，负责接收输入、展示计算过程和结果、调度后台任务。整个软件采用三栏布局，左边输数据、中间看关系图、右边看排序结果，所有耗时的计算都在后台线程跑，不会卡住界面。模块内部按照"视图-控制器"分层，界面组件只负责显示和收事件，逻辑都放在 MainController 里，方便对接其他模块。

二 主窗口与布局设计

MainFrame 是主窗口，从上到下是工具栏、三栏分割面板、底部状态栏。中间用 JSplitPane 水平分成三块，宽度可以拖拽调整，窗口拉大拉小的时候自动适配。顶部工具栏放最常用的按钮：打开文件、清空、开始计算、取消计算、导出结果、导出图片、切换布局，按钮大小、颜色、圆角都统一从 UIStyle 取。底部状态栏左边显示当前有多少节点、多少条边、多少个排序结果，右边显示当前状态提示，比如"正在计算..."或者"计算完成，共 X 条结果"。

左边的 InputPanel 就是一个多行文本框，用户可以直接往里贴关系数据，下面放三个小按钮：导入文件、清空、加载示例。右边的 ResultPanel 用 JList 显示所有拓扑排序结果，做了分页，每页只显示 20 条，下面有翻页和复制按钮，就算有几千条结果也不会一次性把界面卡死。

三 主控制器任务调度设计

MainController 是整个 GUI 层的中间层，界面上点什么都先交给它，它再去调算法、调可视化、调文件读写。所有耗时操作都通过 SwingWorker 放到后台线程跑，不阻塞事件 dispatch 线程，用户界面点什么都会有反应。

用户点"计算"之后，控制器先检查输入是不是空的，然后启动后台任务：第一步先调用 D 的解析模块把文本转成图，第二步调用 A 的全枚举算法算所有拓扑序。计算过程中控制器一直在监听取消按钮，用户点取消就设置中断标记，算法每递归一层就检查一下标记，及时停下来。算完了回到界面线程，把图传给画布组件把结果画出来，把排序列表传给结果面板分页显示，同时把状态栏的统计数字更新好。

除了计算流程，控制器还负责跨模块联动：用户在结果列表里点某一行，控制器把这一行的序列和序号传给画布，画布就高亮这条路径；用户点导出，控制器把结果列表和"有没有算完、为什么停"的信息传给文件模块，写到文件头里。所有报错都通过统一弹窗弹出来，带上具体是第几行格式错了，方便用户找问题。

四 结果面板与交互设计

ResultPanel 内部存了完整的结果列表和当前页码，每次翻页只取当前页的 20 条塞到 JList 里。列表里每条结果用 -> 把课程连起来，和界面显示、导出文件的格式是一样的。

用户单击某一行结果的时候，面板通过监听器把这条序列传给控制器，控制器再让画布高亮。为了让用户能区分不同结果，画布上按结果序号循环用五种颜色高亮（绿、橙、紫、青、粉），节点右上角还标上顺序号，用户切不同结果的时候一眼就能看到路径变了。用户双击某一行，直接把这条结果拼成字符串复制到剪贴板，不用自己抄。面板上面显示结果总数和是否完整的提示，下面显示当前第几页、共多少页，配翻页按钮。

五 界面样式统一设计

UIStyle 是一个工具类，专门管整个界面的配色、字体和按钮样式。工具栏上的按钮、画布上悬浮的缩放按钮，都通过同一个方法配置，圆角、边框、鼠标移上去的效果都一样。主色是低饱和度的蓝色，正常是浅蓝底，鼠标移上去加深，和整体浅色主题配着很舒服。

开发的时候发现 Windows 下 Swing 显示特殊 Unicode 符号（比如那种循环箭头、放大镜图标）会变成方框，最后干脆所有按钮上都用中文写，比如"重置"，再也不乱码。字体统一用系统默认的无衬线字体，正文和按钮字大小差不多，状态栏和提示文字稍小一号，视觉层次清楚。所有弹窗的标题、图标、边框颜色也统一，错误提示红边，普通提示蓝边，交互风格一致。

我也发现自己容易在记录中保留过多细节，最终整理时把过程说明、详细设计和个人感想写得相互重复。完整的验证材料有保存价值，但面向不同读者时，需要分别突出设计、证据和个人认识，不必在每一份文档中重复同样的测试数字。今后我会在阶段工作完成后及时归纳结论，把原始输出留在验证材料中，让报告更简洁，也让真正重要的收获更容易被看见。

在测试和时间安排上，我已经积累了一些自动化自测，但跨模块检查仍有不少步骤依赖临时组合与手动复核。后续可以将高频回归场景整理成固定入口，尤其关注取消、连续计算以及导出状态的一致性，并在记录中保留版本和参数。我也会更早预留文档核对与交付检查的时间，避免代码推进后，运行说明和报告仍停留在旧状态。

六 小结

这次实践让我从“把算法写出来”进一步认识到“让算法成为可靠软件的一部分”需要哪些工作。我既加深了对图结构、拓扑排序和枚举限制的理解，也学会了用明确的接口、具体的反馈和可复现的验证与组员合作。今后面对类似项目，我会继续重视实现质量，同时更主动地考虑调用者、使用者和协作成员的需要，把这些经验用于后续的学习与开发。